



Guía ejercicios 1.1.2

Guía de ejercicios: Python → Java y Java básico

Parte I — Traducción de Python a Java

En cada ejercicio se entrega un algoritmo funcional en Python. Debes:

1. Comprender qué hace el algoritmo.
2. Traducirlo a Java manteniendo su comportamiento.
3. Probarlo con distintos valores de entrada.
4. No modificar la lógica del algoritmo salvo que sea necesario por las diferencias entre ambos lenguajes.



Algunos ejercicios de la guía solicitan ingresar datos por consola. Para ellos puedes buscar en la documentación oficial la clase Scanner y averiguar cómo utilizarla. **NO USES IA**, no atrofies tu aprendizaje. **LEE, PRUEBA , FALLA E INVESTIGA.**



Adicionalmente, puede evitar el uso de ingreso de data por consola entregando los valores en duro en el código. Por ejemplo:

Si en Python te piden esto:

```
numero = int(input("Ingrese un número: "))
```

Y en Java no quieres implementar Scanner aún, puedes hacer simplemente:

```
int numero = 6;
```

Y ejecutar el programa.

Ejercicio 1 — Clasificador de números

Traduce el siguiente programa de Python a Java:

```
numero = int(input("Ingrese un número: "))

if numero > 0:
    if numero % 2 == 0:
        print("Positivo y par")
    else:
        print("Positivo e impar")
elif numero < 0:
    if numero % 2 == 0:
        print("Negativo y par")
    else:
        print("Negativo e impar")
else:
    print("El número es cero")
```

El programa Java debe producir exactamente el mismo resultado.

Prueba al menos con:

```
10
-7
-8
```

```
0
15
```

Después de realizar la traducción, modifica el programa para que además indique si el número es divisible por 3.

Ejercicio 2 — Calculadora de tarifas

Traduce a Java el siguiente programa:

```
cantidad = int(input("Ingrese la cantidad: "))
tipo_cliente = int(input("Ingrese tipo de cliente (1, 2 o 3): "))

if cantidad <= 10:
    costo = cantidad * 500
elif cantidad <= 20:
    costo = 10 * 500 + (cantidad - 10) * 400
else:
    costo = 10 * 500 + 10 * 400 + (cantidad - 20) * 300

if tipo_cliente == 1:
    descuento = 0
elif tipo_cliente == 2:
    descuento = costo * 0.15
else:
    descuento = costo * 0.20

total = costo - descuento

print("Costo: ", costo)
print("Descuento: ", descuento)
print("Total: ", total)
```

Traduce el algoritmo respetando la estructura original.

Luego modifica la versión Java para que:

- No permita cantidades negativas.

- No permita tipos de cliente distintos de 1, 2 o 3.
- Muestre el total redondeado a dos decimales.

Presta especial atención a los operadores `*`, `/`, `-`, `<=`, `==` y a la precedencia de las operaciones.

Ejercicio 3 — Análisis de una secuencia

Traduce a Java:

```
cantidad = int(input("¿Cuántos números ingresará?: "))

positivos = 0
negativos = 0
ceros = 0
pares = 0
impares = 0
suma = 0

for i in range(cantidad):
    numero = int(input("Ingrese un número: "))

    suma += numero

    if numero > 0:
        positivos += 1
    elif numero < 0:
        negativos += 1
    else:
        ceros += 1

    if numero % 2 == 0:
        pares += 1
    else:
        impares += 1

promedio = suma / cantidad

print("Positivos:", positivos)
```

```
print("Negativos:", negativos)
print("Ceros:", ceros)
print("Pares:", pares)
print("Impares:", impares)
print("Suma:", suma)
print("Promedio:", promedio)
```

Después de traducirlo, agrega a la versión Java:

- El número mayor.
- El número menor.
- Cuántos números fueron mayores que el promedio.

No puedes almacenar todos los números en un arreglo para resolver el mayor y el menor.

Debes resolverlo mediante variables auxiliares, actualizándolas durante la iteración.

Ejercicio 4 — Número primo

Traduce a Java el siguiente algoritmo:

```
numero = int(input("Ingrese un número: "))

es_primo = True

if numero < 2:
    es_primo = False
else:
    divisor = 2

    while divisor < numero:
        if numero % divisor == 0:
            es_primo = False
            break

        divisor += 1
```

```
if es_primo:
    print("El número es primo")
else:
    print("El número no es primo")
```

Prueba con:

```
1
2
7
15
17
25
97
100
```

Después de traducirlo, modifica el algoritmo para que solamente pruebe divisores hasta que:

```
divisor * divisor > numero
```

Compara ambas versiones.

Finalmente, crea una versión que muestre todos los números primos entre 1 y 100.

Ejercicio 5 — Frecuencia de números

Traduce el siguiente programa Python a Java:

```
numeros = {}

cantidad = int(input("¿Cuántos números ingresará?: "))

for i in range(cantidad):
    numero = int(input("Ingresa un número: "))

    if numero in numeros:
        numeros[numero] += 1
    else:
```

```
    numeros[numero] = 1

print(numeros)
```

La versión Java debe utilizar un `HashMap<Integer, Integer>`.

Luego modifica el programa para mostrar:

- El número que más veces aparece.
- La cantidad de veces que aparece.
- El número que menos veces aparece.
- La cantidad de números distintos ingresados.

Ejemplo:

```
Entrada:
4
2
5
2
2
2
5
```

Resultado esperado:

```
Número más frecuente: 2
Frecuencia: 3
Número menos frecuente: 5
Frecuencia: 2
Números distintos: 2
```

Presta especial atención a la traducción conceptual:

```
if numero in numeros:
```

y a su equivalente utilizando `HashMap`.

Parte II — Java básico

En estos ejercicios ya no se entrega una solución en Python. Debes diseñar y programar directamente la solución en Java.

Ejercicio 6 — Calculadora de operaciones

Crea un programa Java que solicite dos números y permita seleccionar una operación:

- ```
1. Suma
2. Resta
3. Multiplicación
4. División
5. Módulo
```

El programa debe mostrar el resultado de la operación seleccionada.

Debes controlar:

- Opción inválida.
- División por cero.
- Módulo por cero.

Ejemplo:

```
Primer número: 15
Segundo número: 4

Seleccione una operación:
1. Suma
2. Resta
3. Multiplicación
4. División
5. Módulo

Opción: 5

Resultado: 3
```

Utiliza `switch` para resolver la selección de la operación.

Después agrega una opción que permita determinar si el primer número es divisible exactamente por el segundo.

---

## Ejercicio 7 — Cajero automático

Crea un programa que simule un cajero automático.

El saldo inicial es:

```
$100000
```

El usuario puede realizar las siguientes operaciones:

1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir

El programa debe continuar funcionando hasta que el usuario seleccione salir.

Al retirar dinero:

- El monto debe ser positivo.
- No se puede retirar más dinero que el saldo disponible.

Al depositar:

- El monto debe ser positivo.

Ejemplo:

```
Saldo: $100000

1. Consultar saldo
2. Depositar
3. Retirar
4. Salir

Opción: 3
Monto: 25000

Retiro exitoso.
Saldo actual: $75000
```

Debes utilizar un ciclo `while` y decisiones para controlar las distintas situaciones.

## Ejercicio 8 — Secuencia de números

Solicita al usuario un número `N` y genera una secuencia desde `1` hasta `N`.

Para cada número debes determinar:

- Si es par o impar.
- Si es divisible por 3.
- Si es divisible por 5.

Además, debes llevar los siguientes contadores:

- Cantidad de pares.
- Cantidad de impares.
- Cantidad de números divisibles por 3.
- Cantidad de números divisibles por 5.

Ejemplo para `N = 10`:

```
1 → impar
2 → par
3 → impar, divisible por 3
4 → par
5 → impar, divisible por 5
6 → par, divisible por 3
...
```

Al finalizar:

```
Pares: 5
Impares: 5
Divisibles por 3: 3
Divisibles por 5: 2
```

Después modifica el programa para que indique cuántos números son simultáneamente divisibles por 3 y 5.

Finalmente, modifica el algoritmo para que termine inmediatamente si el usuario ingresa un `N` menor que 1.

## Ejercicio 9 — Adivina el número

Crea un programa que genere un número secreto entre 1 y 100.

El usuario debe intentar adivinarlo.

Después de cada intento, el programa debe indicar:

```
El número secreto es mayor.
```

O:

```
El número secreto es menor.
```

Cuando el usuario acierte:

```
¡Correcto!
Intentos: 7
```

El programa debe continuar solicitando números hasta que el usuario encuentre el número secreto.

Además:

- No debes contar entradas inválidas como intentos.
- Si el usuario ingresa un número menor que 1 o mayor que 100, debe solicitar nuevamente el número.
- Al finalizar, indica si consiguió acertar en menos de 5 intentos.

## Desafío

Después de resolverlo, agrega una opción para que el usuario pueda jugar nuevamente sin reiniciar el programa.

---

## Ejercicio 10 — Sistema de análisis de notas

Crea un programa Java que permita registrar las notas de un grupo de estudiantes.

Primero solicita la cantidad de estudiantes.

Para cada estudiante solicita:

```
Nombre
Nota 1
Nota 2
Nota 3
```

Calcula el promedio:

```
promedio = (nota1 + nota2 + nota3) / 3
```

Para cada estudiante muestra:

```
Nombre
Promedio
Aprobado / Reprobado
```

Se aprueba con promedio mayor o igual a `4.0`.

Al finalizar, el programa debe mostrar:

```
Cantidad de aprobados
Cantidad de reprobados
Promedio general del curso
Mayor promedio
Nombre del estudiante con mayor promedio
Menor promedio
Nombre del estudiante con menor promedio
Cantidad de estudiantes con promedio >= 6.0
```

No puedes ordenar los estudiantes para encontrar el mayor y menor promedio.

Debes resolverlo mediante variables auxiliares y comparaciones durante el procesamiento.

## Desafío final

Agrega un menú:

```
1. Registrar estudiantes
2. Mostrar estadísticas
3. Buscar estudiante
4. Salir
```

La opción "Buscar estudiante" debe solicitar un nombre y determinar si existe dentro de los estudiantes registrados.

Si existe, muestra su promedio y situación académica.

Para almacenar los datos puedes utilizar arreglos o `ArrayList`.